

DashVMC

Real-Time Discrete World Model Control in Geometry Dash

Contributors

Florent Tardiolle¹, Florian Yger^{1,2}

¹INSA Rouen Normandy ²LITIS

World-model agents are usually evaluated in simulators that can wait for the policy; live games impose the opposite constraint, requiring capture, prediction, and action before the next frame. We present *DashVMC*, which learns a compact, action-conditioned world model from approximately two hours of recorded Geometry Dash gameplay. To test whether the learned dynamics are actionable, a controller is initialized by behavioural cloning (BC) and refined with Proximal Policy Optimization (PPO) entirely in frozen-model rollouts, without further interaction with the live game. Across three controller seeds, the refined policies improve mean live survival over their BC initializations on all three official levels and also improve on a held-out community layout. At deployment, the controller reuses the world model’s temporal summary while skipping visual generation; the complete capture-to-action path sustains a 60-Hz loop on a consumer GPU. Action-conditioned continuations and rollout diagnostics show that the model remains useful for control despite imperfect long-horizon fidelity.

Project page: tardiolle.github.io/dash-vmc

Code: github.com/Tardiolle/dash-vmc

Correspondence: florent.tardiolle@insa-rouen.fr

1 Introduction

A world model learns transition dynamics $p_\theta(s_{t+1} | s_t, a_t)$ and can be rolled forward so that an agent practices without further environment interaction (LeCun, 2022; Ha and Schmidhuber, 2018). DashVMC asks whether such practice can improve a policy that must then act in a live, non-paused game. It predicts a 64-token next grid in parallel for inexpensive imagined rollouts, then skips generation and reuses the transformer’s temporal summary during deployment.

Most world-model agents are evaluated in synchronous environments that wait for the policy. A live, non-paused application reverses that contract: screen capture, perception, inference, and action dispatch must finish before the action becomes stale.

Geometry Dash is a small but unforgiving test of this contract. The avatar moves continuously, the action space is binary, and a mistimed jump causes immediate death. Levels are deterministic, but the agent observes only captured pixels and acts through keyboard events; it does not receive emulator state during control. The apparent simplicity therefore isolates a demanding question: can an agent learn when to act from pixels, improve without touching the live game, and still react within narrow timing windows on modest hardware?

DashVMC follows the perception–dynamics–control organization of World Models (Ha and Schmidhuber, 2018): it compresses each frame into a discrete grid, learns action-conditioned grid dynamics, initializes a lightweight action model with BC, and refines it with PPO (Schulman et al., 2017) in the frozen model.

Our contributions are:

1. a compact discrete, action-conditioned world model learned from approximately two hours of offline gameplay, whose rollouts support policy refinement without further live-game interaction;

2. a decoder-free deployment path that reuses the model’s temporal summary and sustains a 60-Hz capture-to-action loop on a consumer GPU; and
3. evidence that the learned dynamics are actionable: controllers refined only in frozen-model rollouts improve over their exact BC parents on all three official levels across three seeds and on a held-out layout.

The contribution is the integrated real-time system and its evaluation rather than a new RL algorithm; diagnostics cover action sensitivity, rollout fidelity, targeted loss ablations, and latency.

2 Related work

World models and imagination. World Models demonstrated that a compact controller optimized in learned dreams can transfer to the real environment (Ha and Schmidhuber, 2018). IRIS combines a discrete autoencoder with an autoregressive transformer for data-efficient Atari learning (Micheli et al., 2023); Δ -IRIS reduces sequence cost with separate change and context tokens (Micheli et al., 2024); DIAMOND uses diffusion for imagination-trained control (Alonso et al., 2024); and DreamerV3 scales categorical recurrent latents across domains (Hafner et al., 2025). TWISTER adds action-conditioned CPC to transformer dynamics (Burchi and Timofte, 2025), which DashVMC adopts alongside parallel spatial prediction. Earlier screenshot-based Geometry Dash control used DQN and imitation under a fabricated timestep (Li and Rafferty, 2017); here the game clock continues during capture and inference.

FSQ tokenization and policy initialization. FSQ replaces learned vector-quantization codebooks with bounded scalar levels, avoiding commitment losses and codebook collapse while exposing an integer coordinate lattice (Mentzer et al., 2024); we use the improved iFSQ bounding function (Lin et al., 2026). Demonstration-based RL can reduce costly early exploration (Hester et al., 2018; Baker et al., 2022); here recorded actions initialize the controller, while the retained policy is selected only after imagined PPO refinement. Deterministic Sobel preprocessing removes appearance detail before tokenization, providing a compact task-specific alternative to richer continuous or generative representations.

3 DashVMC

DashVMC is organized around a simple loop: recorded play trains a world model, the action model is refined in rollouts from the frozen dynamics model, and the resulting controller is returned to the live game. The visual encoder, dynamics model, and action model are trained in sequence and then frozen for deployment. Figure 1 shows how the same learned state supports both imagined practice and live reaction.

3.1 Visual tokenization

Each RGB capture is cropped, converted to grayscale, filtered with Sobel edges, and resized to 64×64 . A convolutional autoencoder maps the result through three stride-2 stages to a four-channel 8×8 latent map. Each spatial vector is independently quantized with iFSQ levels $[8, 5, 5, 5]$, using the bounding function $2 \text{sigmoid}(1.6z) - 1$, and yields 64 token IDs from 1000 implicit codes (Mentzer et al., 2024; Lin et al., 2026). The tokenizer is trained on adjacent frames with reconstruction, temporal-slowdown, and latent-uniformity objectives (Xia et al., 2025). Once selected, its encoder is frozen; its decoder is used to visualize dreams but is not required by the action model.

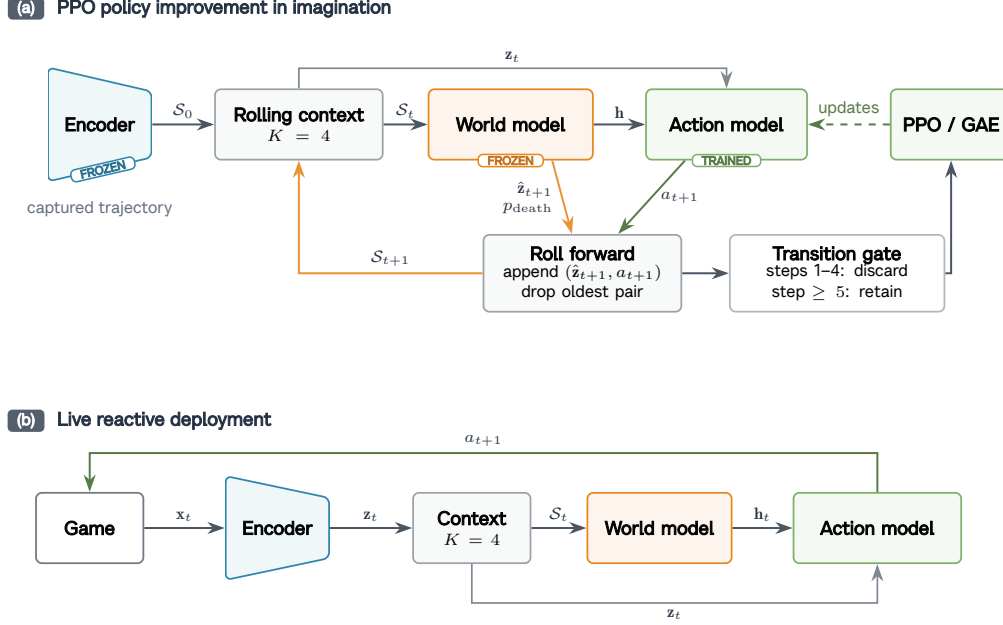


Figure 1 Learning in imagination and acting in the live game. **(a)** A recorded context seeds the frozen world model; the action model chooses what to do, the world model predicts the consequence, and PPO updates only the action model. **(b)** In the live game, the encoder and temporal context provide the state used for action selection, while visual generation is skipped.

3.2 Action-conditioned dynamics

The dynamics model is an eight-layer, width-384 transformer that receives four recent frame–action pairs. Each frame block contains 64 visual tokens and one ALIVE/DEATH status token, with action tokens inserted between blocks; a final masked block requests the next frame. Attention is bidirectional within a frame and causal across frame/action blocks, so all next-frame positions are predicted in one forward pass without seeing their targets. An explicit action token separates consecutive frames, so the model can answer the question central to policy learning: what changes if the action model jumps rather than idles? It predicts all 64 codes of the next grid in parallel, together with an ALIVE/DEATH score, and exposes a temporal summary $\mathbf{h}_t$, taken from the most recent action-token position, to the action model. Parallel prediction keeps imagined steps inexpensive, while the rolling context lets generated consequences become the starting point for the next decision.

Training combines next-grid prediction with action-conditional contrastive predictive coding of weight 0.1 (Burchi and Timofte, 2025; van den Oord et al., 2018), context corruption, and an auxiliary death objective. For context corruption, 5% of context tokens are replaced uniformly and another 5% with adjacent FSQ codes. The token loss uses focal modulation (Lin et al., 2017) and structured label smoothing (SLS): for target lattice coordinate $\mathbf{c}_i$, smoothing mass $\varepsilon = 0.1$ is distributed over non-target codes proportionally to $\exp(-\|\mathbf{c}_i - \mathbf{c}_j\|_2^2 / 2\sigma^2)$, with $\sigma = 0.9$. Focal modulation is applied once to the aggregate soft-target cross-entropy, and the output projection is tied to the token embedding. This is a task-specific design choice rather than a general claim about world-model training.

3.3 Controller and latent policy improvement

The 45,546-parameter actor–critic embeds the current token grid, processes it with two convolutions and max pooling, and concatenates the resulting 256-dimensional feature with the dynamics model’s temporal summary $\mathbf{h}_t$. Separate heads predict jump probability, value, and an eight-action auxiliary output covering the current and next seven actions. BC first reproduces recorded action timing; PPO then departs from imitation through repeated imagined consequences.

Each PPO iteration generates 512 imagined episodes of at most 45 steps from recorded four-frame contexts, with greedy next-grid predictions and sampled Bernoulli actions. The first four generated transitions refresh the context but are excluded from PPO because the recorded approach may already make a collision unavoidable. The remaining horizon covers the obstacles visible in the seed while avoiding long updates dominated by geometry invented beyond it. Rollouts terminate when the death logit exceeds the ALIVE logit; because death frames are oversampled $5\times$, this is an operational boundary rather than a calibrated probability. Reward is $+1$ per survived step and -0.25 per jump, discouraging high-duty-cycle jumping. We use Generalized Advantage Estimation (GAE) (Schulman et al., 2015) with $\gamma = 0.995$, $\lambda = 0.95$, PPO clip 0.2, and auxiliary-action loss weight 0.1.

3.4 Reactive deployment path

Training asks the world model to generate consequences; live control only asks it to interpret what has just happened. Desktop Duplication captures only the 1032×1032 model crop; CPU grayscale conversion followed by CUDA Sobel filtering and area resizing produces the 64×64 observation. Each frame is encoded and appended to a GPU-resident context, and the transformer runs context prefill only to produce $\mathbf{h}_t$. The live context marks every observation ALIVE; process-memory death detection only terminates and restarts an attempt outside the controller. The next-grid predictor and image decoder are skipped, reducing the deployed path to 15.57 million parameters; the encoder uses `torch.compile`, and transformer context encoding uses a CUDA Graph after warm-up. The accelerated preprocessing path was checked against the recorded-data pipeline on synthetic and fresh live captures and produced byte-identical observations. This is a reactive controller rather than an online planner: the policy has already performed its search through experience during offline imagined training.

4 Experimental setup

Data and training. Before augmentation, the corpus contains 4,264 action-labelled gameplay episodes, or 251,963 frames and approximately 2.33 hours at the nominal 30-FPS capture cadence. It includes deliberate failures, official Levels 1–7, several community levels, and 36 expert trajectories or segments totaling less than 20 minutes. BC uses action labels from both the deliberate-failure and expert corpora. During recording, the current key state is sampled immediately after each capture. During deployment, the previously held state is appended with the new capture, after which the controller dispatches the state held until the next tick; process memory only segments deaths and restarts. Mechanics outside the selected scope, including gravity inversion, are excluded. Failure and expert trajectories are each split 90/10 with seed 42 at the base-episode level. During tokenizer training, each adjacent-frame pair receives a shared two-dimensional translation whose horizontal and vertical offsets are sampled independently and uniformly from $[-4, 4]$ pixels. For dynamics training, the frozen encoder instead tokenizes each base episode at vertical offsets $\{-4, -2, 0, 2, 4\}$, expanding the corpus to 21,320 episode variants. All windows and variants inherit their base-episode assignment; although no complete trajectories are duplicated, deterministic repeated attempts can share near-identical prefixes, so the 10% stratum is an in-distribution development set rather than a generalization test. The tokenizer, dynamics model, and BC controller use this split; PPO draws rollout seeds from the full corpus and uses 512 fixed development contexts for checkpoint selection. Tokenizer and dynamics training use seed 42, while controller training is repeated with seeds 43–45 under the same frozen representation and world model. The first two stages run sequentially in BF16 on one A100, controller training runs in BF16 on one H200 at a time, and live inference and local latency evaluation use the same RTX 2060 with PyTorch 2.11.0+cu126 (CUDA 12.6).

The retained full runs took 3.19 A100 hours for the tokenizer and 4.20 A100 hours for the dynamics model; the seed-43 $H = 45$ PPO baseline took 17.41 H200 hours for 15,000 iterations.

Checkpoint and live evaluation. PPO checkpoint selection uses survival on 512 fixed latent development contexts every ten iterations. All live checkpoints are then frozen and act deterministically from pixels alone; Auto-Retry standardizes restarts but provides no policy input. For each seed, we compare PPO with the exact BC checkpoint that initialized it over 25 scored attempts on *Stereo Madness*, *Back on Track*, and *Polargeist*. The BC and PPO attempts are separate samples paired only by controller lineage, and every

Table 1 Training and checkpoint selection; tokenizer/dynamics learning rates use cosine decay.

Stage	Optimization	Selection
Tokenizer	Adam; 1,000 epochs; batch 2,048; LR $10^{-3} \rightarrow 10^{-5}$; slowness 0.1; uniformity 0.01.	Min. loss (epoch 920).
Dynamics	AdamW; 200×500 steps; batch 512; LR $2 \times 10^{-3} \rightarrow 5 \times 10^{-5}$; weight decay 0.01; dropout 0.1; death oversampling 5×.	Max. death F1 (epoch 139).
BC	Three seeds; AdamW; 50 epochs/seed; batch 512; LR 10^{-3} ; weight decay 10^{-4} ; positive jump weight 1.5.	Min. loss (10; 10; 11).
PPO	Three seeds; Adam; 15,000 iterations/seed; 512 rollouts/iteration; four update epochs; minibatch 512; LR 10^{-4} ; entropy/critic weights 0.01/0.5; ratio clip 0.2; max grad. norm 0.5.	Max. survival (12,420; 5,090; 12,280).

attempt runs until detected death without right-censoring. Although policy weights and level geometry are fixed, the background phase and the scene’s hue and saturation cycle independently of level progress, producing a different pixel-level observation stream on each attempt. Capture scheduling and keyboard-dispatch phase add further variation around narrow timing boundaries; repeated attempts measure this deployment variability. The same protocol is used on two auxiliary layouts. *Stereo Madness Copy* begins at the official level’s first ship section, revealing later control behaviour that the official Level 1–3 runs do not reach. *Stereo INSANE Nerfed* is a community layout first accessed after the corpus and checkpoints were frozen, providing a held-out test within the supported mechanics. To validate the earlier 30-FPS evaluations, a cadence check compares the identical frozen PPO checkpoint on Stereo Madness Copy over 25 optimized-path attempts at 60 FPS and 20 earlier diagnostic-path attempts at 30 FPS.

The primary endpoint is frames survived; the cross-cadence probe uses episode wall time because frame counts change with cadence. PPO-minus-parent-BC intervals independently resample the 25 attempts from each frozen policy 50,000 times. These attempt-level intervals characterize deployment variability rather than controller-training uncertainty; the three controller-seed differences provide the replication evidence.

A post-freeze probe retrains seed 43 from the same BC parent with $H = 20$ for 30,000 iterations (15.61 H200 hours), versus $H = 45$ for 15,000 (17.41 hours), and scores 10 versus 25 attempts per layout. Because horizon and iteration count co-vary, this is a roughly compute-matched shorter-rollout probe, not an isolated ablation.

Latency and continuation protocols. Live latency is measured from capture through action dispatch over 5,000 consecutive frames of the optimized 60-FPS path. For the qualitative intervention, the world model is initialized from a fixed encoded four-frame prefix immediately before a spike. Greedy continuations differ only in the first conditioned action, jump or idle; all subsequent actions are idle. Post-hoc quantitative diagnostics re-encode the unaugmented base trajectories in the fixed development stratum with the selected tokenizer. They contain 22,745 next-frame windows from 413 usable trajectories; twelve additional failure trajectories are shorter than the five frames required for one context–target window. For the paired action intervention, we hold each recorded four-frame context fixed and flip only the final binary action, which directly precedes the target frame. Its interval resamples whole trajectories 5,000 times; autoregressive diagnostics feed predicted grids back greedily while replaying recorded actions. The standard cohort contains 1,024 starts, capped at 64 per trajectory, from 150 failure and three expert trajectories; an exploratory 200-step cohort contains 256 starts from the only four sufficiently long development trajectories.

Targeted dynamics-loss ablations. We compare the reported structured-SLS+CPC dynamics model with three single-training-run variants: uniform label smoothing with the same $\varepsilon = 0.1$, no label smoothing, and structured SLS without CPC. They retain the tokenizer, corpus, split, architecture, seed 42, optimization, and maximum-development-death-F1 selection rule; all checkpoints share the exact evaluation windows and rollout starts.

External transition-latency protocol. We also time the native imagined transition of DashVMC and released Pong checkpoints of IRIS and DIAMOND on the same RTX 2060. Each runs in a fresh process at

batch one with FP32 eager execution for 30 warm-ups and two repetitions of 100 synchronized transitions; differing native interfaces make this a cost, not quality, comparison.

5 Results

5.1 Policy refinement in the live game

The central question is whether policy improvement learned only through frozen-model rollouts transfers beyond imitation to the live game. The selected tokenizer reconstructs development edge maps at 34.10 dB PSNR, and the dynamics model reaches 29.58% visual-token accuracy. Deaths comprise 1.80% of its 22,745 development windows; at threshold 0.5 the death head obtains 0.721 precision, 0.883 recall, 0.794 F1, 0.994 AUROC, and 0.050 expected calibration error. Because this stratum selected the dynamics checkpoint, these are in-distribution development diagnostics rather than untouched test results. These component metrics establish a usable but imperfect imagined environment; the live comparison below tests whether it nevertheless teaches better actions.

Table 2 Live survival on all five layouts. No-op entries are mean frames $\pm$ standard deviation over 10 attempts and are shared across seeds. BC/PPO entries use 25 attempts; Δ is PPO minus its exact parent BC [95% attempt-level bootstrap interval]. The first three are official levels, Stereo Madness Copy begins in ship form, and Stereo INSANE Nerfed is held out.

Level	Policy	Seed 43	Seed 44	Seed 45
Stereo Madness	No-op	shared: 46.2 ± 0.7		
	BC	120.2 ± 91.8	130.6 ± 76.0	156.0 ± 100.2
	PPO	280.1 ± 23.3	280.4 ± 33.5	308.8 ± 68.6
	Δ [95% CI]	159.9 [122.7, 195.4]	149.8 [116.8, 180.6]	152.8 [106.7, 199.8]
Back on Track	No-op	shared: 63.4 ± 0.7		
	BC	123.5 ± 81.5	111.1 ± 60.3	120.2 ± 69.6
	PPO	260.0 ± 55.5	195.9 ± 126.5	209.4 ± 130.0
	Δ [95% CI]	136.5 [98.8, 174.4]	84.8 [31.9, 139.6]	89.2 [34.6, 147.2]
Polargeist	No-op	shared: 41.5 ± 0.7		
	BC	52.4 ± 16.7	49.0 ± 16.5	46.2 ± 9.2
	PPO	65.2 ± 33.4	68.3 ± 44.1	63.1 ± 32.9
	Δ [95% CI]	12.8 [−0.2, 28.6]	19.4 [3.0, 39.0]	16.8 [5.7, 31.5]
Stereo Madness Copy	No-op	shared: 135.0 ± 0.0		
	BC	366.8 ± 172.8	285.7 ± 153.5	299.4 ± 175.2
	PPO	435.6 ± 196.1	495.8 ± 137.5	546.8 ± 94.1
	Δ [95% CI]	68.8 [−33.3, 166.1]	210.1 [127.9, 285.5]	247.4 [167.6, 320.7]
Stereo INSANE Nerfed	No-op	shared: 46.1 ± 0.8		
	BC	104.8 ± 72.1	102.1 ± 64.0	97.8 ± 57.9
	PPO	290.4 ± 48.6	342.8 ± 80.8	266.7 ± 128.9
	Δ [95% CI]	185.5 [151.1, 217.6]	240.6 [200.8, 280.2]	168.8 [115.4, 223.9]

The shared no-op rows of Table 2 establish the level-dependent difficulty floor: BC exceeds no-op on every layout, and PPO then improves over its parent imitation policy in all fifteen seed-level comparisons. Across the three controller seeds, the paired improvements are 154.2 ± 5.2 frames on *Stereo Madness*, 103.5 ± 28.7 on *Back on Track*, and 16.3 ± 3.3 on *Polargeist*, where the variation is the sample standard deviation across the three seed differences. Every attempt-level interval on the first two levels excludes zero. The smaller Polargeist improvement is positive for all three seeds, although seed 43’s deployment-variability interval marginally includes zero; seeds 44 and 45 exclude it. Polargeist introduces a new timing demand—yellow orbs that require a second input while airborne—which helps explain why improvement is smaller there.

On Stereo Madness Copy, seed 43’s attempt-level interval includes zero while seeds 44 and 45 exclude it; the mean paired improvement across seeds is 175.4 ± 94.2 frames. On the post-freeze Stereo INSANE Nerfed layout, all three intervals exclude zero and the mean paired improvement is 198.3 ± 37.6 frames, providing the paper’s generalization evidence within the supported mechanics. Stereo Madness Copy is not held out, but

it exposes a different qualitative ability: starting at the official level’s first ship section, the agent maintains altitude and avoids obstacles with strong ship control. The 60-FPS cadence probe on Stereo Madness Copy yields similar observed real-time survival: mean recorded episode wall time is 17.72 s at 60 FPS versus 17.79 s at 30 FPS, with a 95% bootstrap interval of $[-1.04, 0.89]$ s for the mean difference (60 FPS minus 30 FPS). This limited one-checkpoint, one-layout probe supports cadence transfer there, not equivalence across the primary cube levels.

5.2 Live control at game speed

Reactive play requires the complete perception-to-action loop, not just policy inference, to finish within each 16.67-ms frame. At 60 Hz, the controller can revise the binary key state every frame—60 decisions per second, not 60 distinct key presses. While the RTX 2060 also renders the game, 5,000 complete capture-to-action measurements have mean/median 12.018/12.083 ms, p95 14.068 ms, p99 14.913 ms, and maximum 24.059 ms. Eleven frames (0.22%) exceed the period; on a miss the loop skips its pacing sleep and immediately begins the next capture rather than reusing an old action. This doubles the temporal resolution of the 30-FPS demonstrations. Qualitatively, the ship policy uses sustained thrust and release intervals: ship motion remains correctable in flight, whereas a cube jump is largely ballistic until landing and therefore demands sparser, more precise timing. The smooth, low-duty-cycle behaviour is consistent with PPO’s per-step jump penalty, although it does not isolate that penalty’s effect.

5.3 Cost of an imagined step

DashVMC’s native imagined transition averages 11.05 ms, compared with 49.30 ms for DIAMOND and 177.83 ms for IRIS, corresponding to $4.5\times$ and $16.1\times$ lower latency, respectively. Only DashVMC fits the 16.67-ms 60-FPS period on this hardware.

Table 3 Batch-1 native imagined-transition latency on an RTX 2060 (synchronized run means in milliseconds). Interfaces differ, so this compares operational cost rather than quality.

Model	Run 1	Run 2	Mean
DashVMC	11.04	11.06	11.05
DIAMOND	50.13	48.46	49.30
IRIS	176.94	178.73	177.83

Parallel grid prediction and latent-space feedback avoid autoregressive token generation and pixel decoding at every imagined step, compounding the savings through PPO. The differing native interfaces make this an operational-cost comparison, not evidence of generation-quality or control superiority.

5.4 Action sensitivity and autoregressive fidelity

The learned dynamics model can be rolled out beyond the 45-step training horizon by feeding each predicted grid and newly selected action back into its context. In interactive rollouts it reproduces recognizable Geometry Dash structure, including scrolling terrain, camera movements, height changes, collisions, and avatar transformations. Some continuations remain coherent for long stretches, while others eventually repeat motifs, empty the scene, or produce impossible geometry. Separate free-running demonstrations include continuations lasting hundreds of generated steps. Video demonstrations of interactive level continuations are available on the [project page](#).

Table 4 Recorded-action rollout diagnostics. Accuracy and decoder-space PSNR compare predicted with recorded grids; JS compares pooled token marginals. Standard: 1,024 starts from 153 trajectories; extended: 256 starts from four long trajectories.

Cohort	Horizon	Token acc. $\uparrow$	PSNR $\uparrow$	Token JS $\downarrow$
Standard	1	30.14%	32.83	0.0037
	5	19.29%	28.25	0.0045
	10	14.06%	25.23	0.0052
	20	8.26%	22.45	0.0079
	45	2.91%	19.99	0.0208
Extended	45	3.02%	19.78	0.0351
	100	1.38%	19.49	0.0428
	200	0.74%	19.39	0.0733

Table 4 is best read relative to screen replacement: scrolling replaces the entire visible screen in roughly 45 frames. The decline from 32.83 dB at horizon 1 to 19.99 dB at 45 therefore marks a transition to fully generated geometry, not only compounding error. Thereafter PSNR is nearly flat within the extended cohort (19.78/19.49/19.39 dB at horizons 45/100/200): long rollouts can remain coherent, but beyond one screen of travel they are not reliable counterfactual environments for PPO.

Action sensitivity is a separate question. Figure 2 tests it while holding the visual context fixed. The two futures begin from the same recorded moment, yet a single changed action produces the expected qualitative divergence: the jump branch clears the spike and lands, while the idle branch collides and terminates. This deliberately short matched-context diagnostic isolates action conditioning; it is not intended to measure long-horizon coherence. The corresponding corpus-wide intervention supports the same conclusion. Across all 22,745 development transitions, the factual action has lower next-frame negative log-likelihood than its flipped counterpart in 69.52% of contexts. The trajectory-mean advantage is 0.165 nats per visual token (95% episode-bootstrap interval [0.150, 0.181]); factual-action token accuracy is 29.58% versus 27.78% after flipping, and 11.04% of argmax token predictions change.



Figure 2 Action-conditioning diagnostic from one shared four-frame context. The rows are aligned at $t + 1$ and $t + 5$: changing only the first future action makes the idle branch collide and terminate, while the jump branch clears the spike and lands by $t + 13$.

Table 5 Matched dynamics-loss ablations on fixed development data. **Ours** uses structured SLS and CPC; other row names denote the changed component. Action advantage is the episode-mean factual-versus-flipped NLL difference. Rollout columns average horizons 1/5/10/20/45. Each row is one run; bold is best.

Variant	Death F1 $\uparrow$	Action adv. $\uparrow$	Mean acc. (%) $\uparrow$	Mean PSNR $\uparrow$	Mean JS $\downarrow$
Ours	.794	.165	14.93	25.75	.0084
No SLS	.808	.196	14.33	25.65	.0115
No CPC	.787	.148	13.89	25.56	.0104
Uniform LS	.826	.156	12.90	25.41	.0141

Table 5 condenses each rollout curve into an unweighted mean over the five standard horizons. Ours has the strongest aggregate rollout profile, leading token accuracy and PSNR while minimizing token-marginal JS. Removing CPC worsens every displayed endpoint relative to Ours. Removing SLS improves death F1 and factual-action advantage but reduces all three rollout aggregates, whereas uniform smoothing attains the highest death F1 but the weakest rollout profile. These single-run, model-level results therefore support CPC and associate FSQ-structured smoothing with better rollout fidelity, but do not establish training variance or live-policy effects without retraining the downstream controller.

Rollout horizon. The roughly compute-matched seed-43 $H = 20$ probe changes macro-average live survival by +1.9% relative to $H = 45$, with a 95% stratified-bootstrap interval of $[-8.1, 13.5]\%$; every per-layout interval includes zero.

Table 6 Seed-43 horizon probe in frames (mean $\pm$ SD). $H = 45$ uses 25 attempts and $H = 20$ uses 10; intervals bootstrap their difference.

Level	$H = 45$	$H = 20$	Δ_{20-45} [95% CI]
Stereo Madness	280.1 $\pm$ 23.3	281.5 $\pm$ 34.9	1.4 [-21.1, 23.8]
Back on Track	260.0 $\pm$ 55.5	319.8 $\pm$ 122.0	59.8 [-19.0, 129.7]
Polargeist	65.2 $\pm$ 33.4	62.7 $\pm$ 29.3	-2.5 [-23.0, 19.9]
Stereo Madness Copy	435.6 $\pm$ 196.1	383.2 $\pm$ 163.6	-52.4 [-173.9, 69.7]
Stereo INSANE Nerfed	290.4 $\pm$ 48.6	296.4 $\pm$ 47.7	6.0 [-27.0, 40.5]

Thus 20-step dreams preserve observed performance and show that late $H = 45$ states are not necessary for this run, but the single seed, unequal attempts, and co-varying iteration count preclude an isolated horizon claim.

6 Limitations

DashVMC studies one deterministic game with binary actions; its post-freeze test is one new layout within known mechanics. The episode split can contain shared or near-identical level prefixes, so model diagnostics are in-distribution development measurements and only the post-freeze layout supports a generalization claim. Three controller seeds share one tokenizer and dynamics checkpoint, measuring controller-training variability rather than end-to-end uncertainty. No matched grid-only or direct frame-stack controller isolates the benefit of reusing $\mathbf{h}_t$; that reuse is an architectural choice, whereas the demonstrated role of the world model is to provide PPO’s offline environment. Death calibration is measured only on recorded states, leaving reward-model exploitation in recursive rollouts unquantified; live transfer validates the endpoint, not the mechanism. Likewise, loss ablations are single-run model diagnostics without controller retraining, and the $H = 20$ probe is single-seed and only roughly compute matched. One-GPU access constrained multi-seed ablations and scaling curves, while wall-clock live evaluation limited deployment tests. Finally, 60-FPS behavioural equivalence is probed on one checkpoint and layout, and reported speed depends on the RTX 2060 and compact edge representation.

7 Conclusion

DashVMC shows that a fixed archive of past gameplay can be turned into an agent that improves without further live interaction and still acts at game speed. From roughly two hours of action-labelled recordings, a compact action-conditioned world model produces action-sensitive, often coherent continuations and serves as an offline practice environment: PPO-refined policies consistently outlive their BC parents when transferred zero-shot to the game. At deployment, the model’s temporal state supports a decoder-free 60-Hz capture-to-action loop on a consumer GPU. Together, these results show that, even in a data-starved setting, a task-specific world model can bridge fixed offline experience, imagined policy improvement, and real-time control.

Acknowledgments

We thank Clément Chatelain and Robin Condat for their guidance during the Representation Learning course at INSA Rouen Normandy, in which this project began, and CRIANN for providing the computing resources used for training. We thank Maël Planchot for contributing gameplay data. Geometry Dash is developed by RobTop Games.

References

- Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (VPT): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv:2206.11795*, 2022.
- Maxime Burchi and Radu Timofte. TWISTER: Learning transformer-based world models with contrastive predictive coding. In *International Conference on Learning Representations*, 2025.
- David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025.
- Todd Hester, Matej Vecerik, Olivier Pietquin, Marc Lanctot, Tom Schaul, Bilal Piot, Dan Horgan, John Quan, Andrew Sendonaris, Gabriel Dulac-Arnold, Ian Osband, John Agapiou, Joel Z. Leibo, and Audrunas Gruslys. Deep q-learning from demonstrations. In *AAAI Conference on Artificial Intelligence*, 2018.
- Yann LeCun. A path towards autonomous machine intelligence. *OpenReview*, 2022. Version 0.9.2.
- Ted Li and Sean Rafferty. Playing Geometry Dash with convolutional neural networks. Stanford CS231N course report, 2017.
- Bin Lin, Zongjian Li, Yuwei Niu, Kaixiong Gong, Yunyang Ge, Yunlong Lin, Mingzhe Zheng, JianWei Zhang, Miles Yang, Zhao Zhong, Liefeng Bo, and Li Yuan. iFSQ: Improving FSQ for image generation with 1 line of code. *arXiv preprint arXiv:2601.17124*, 2026.
- Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *International Conference on Computer Vision*, pages 2980–2988, 2017.
- Fabian Mentzer, David Minnen, Eiríkur Agustsson, and Michael Tschannen. Finite scalar quantization: VQ-VAE made simple. In *International Conference on Learning Representations*, 2024.
- Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023.
- Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. In *International Conference on Machine Learning*, 2024.

- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018.
- Zaishuo Xia, Yukuan Lu, Xinyi Li, Yifan Xu, and Yubei Chen. Clone deterministic 3d worlds with geometrically-regularized world models. *arXiv preprint arXiv:2510.26782*, 2025.